INDEX

S.NO	DATE	NAME OF THE PROGRAM	PAGE.NO	STAFF SIGN
1		Basic unix commands		
2		Shell programming		
3		File system related system calls		
4		Process management		
5		Process Synchronization using semaphores		
6		Inter process communication using shared memory		
7		CPU scheduling		
8		Deadlock detection		
9		Contiguous memory allocation		
10		Page Replacement		

Ex.No:1 **DATE:**

UNIX INTRODUCTION

AIM:

To study the basics of UNIX operating system, commands and vi editor

UNIX Commands

General Commands:

1. **date**: This tells the current date and time.

   $ date

   Thu Oct 15 09:34:50 PST 2005

2. **who**: Gives the details of the user who have logged into the system.

   $ who

   abc tty() oct 15 11:17

   Xyz tty4 oct 15 11:30

3. **whoami**: Gives the details regarding the login time and system's name for the connection being used.

   $ whoami.

   Raghu

4. **man**: It displays the manual page of our terminal with the command 'man' command name.

   $ man who

5. **head and tail**: 'head' is used to display the initial part of the text file and 'tail' is used to display the last part of the file.

   $ head [-count] [filename]

   $ tail [-count] [filename]

6. **pwd:** It displays the full path name for the current directory we are working in.

   $ pwd

/home/Raghu

7. **is** : It displays the list of files in a current working directory.

$ ls
$ ls -l = lists files in long format.
$ ls –t = lists in order of last modification time.
$ ls –a = lists all entries , including the hidden files.
$ ls –d = lists directory files instead of its contents.
$ ls –p = puts a slash after each directory.
$ ls –u = lists in order of last access time.

8. **mkdir**: It is used to create a new directory.

$ mkdir directory name.

9. **cd** : It is used to change from the working directory to any other directory specified.

$ cd changes to home directory.
$ cd.. changes to parent directory.
$ cd / changes to root directory.
$ cd dirl changes to directory dirl.

10. **rmdir**: It is use to remove the directory specified in the command line.

$ rmdir directory name

11. **cat** : This command helps us to specify the contents of the file we specify.

$ cat [option….[file….]]

12. **cp** : This command is used to create duplicate copied of ordinary files.

$ cp file target
$ cp file1 file2 [file1 is copied to file2]

13. **mv**: This command is used to rename and move ordinary and directory files.

$ mv file1 file2
$ mv directory1 directory2

14. **ln**: This is used to link file.

$ ln file1 [file2….] target

15. **rm**: This command is used to remove one or more files from the directory.This can be used to delete all files as well as directory.

    $ rm [option…..] file

16. **chmod**: Change the access permissions of a file or a directory.

    $ chmod mode file

    $ chmod [who] [+/-/=] [permission…] file

    who = a – all users
    g – group
    o – others
    u –user

    [+/-/=] + adds
    - removes
    = assigns

    [permission] r = read
    w =write
    x =execute

    Ex.:: $ chmod 754 prog1.

17. **chown**: change the owner ID of the files or directories.

    Owner may be decimal user ID or a login name found in the file/etc/passwd.

    This utility is governed by the chown kernel authorization. If it is not granted,ownership can only be changed by root.

    Ex:: $ chown tutor test

18. **wc:**counts and displays the lines,words and characters in the files specified.

    $ wc
    $ wc
    $ wc
    $ wc

    Ex:: $ wc prog2.
    60 prog2.

19. **grep :** searches the file for the pattern.

    $ grep [option…] pattern [file….].

    Display the lines containing the pattern on the standard output.

    $ grep c report only the number of matching lines.

    $ grep-l list only the names of files containing pattern.

$grep-v display all lines expert those containing pattern.
Ex: grep c"the"prog2.

20. cut: cuts out selected fields of each line of a file.
$ cut -first [-d char] [file1 file2…].
-d = it is delimiter. Default is tab.

Ex: cut –f1, 3 –d"w"prog2.

21. paste: merges the corresponding lines of the given files.
$ paste –d file1 file2
Option – d allows replacing tab character by one or more alternate characters.
Ex:: paste prog1 prog2

22. sort : arranges lines in alphabetic or numeric order.
$ sort [option]file.
Option –d dictionary order
Option –n arithmetic order
Option –r reverse order
Ex :: $ ls – l | sort –n

Output:

```
telnet@SOMU:~$ date
Sat Jun  1 13:02:04 IST 2024
telnet@SOMU:~$ who
telnet@SOMU:~$ whoami
telnet
telnet@SOMU:~$ man who
telnet@SOMU:~$
```

```
NAME
       who - show who is logged on

SYNOPSIS
       who [OPTION]... [ FILE | ARG1 ARG2 ]

DESCRIPTION
       Print information about users who are currently logged in.

       -a, --all
              same as -b -d --login -p -r -t -T -u

       -b, --boot
              time of last system boot

       -d, --dead
              print dead processes

       -H, --heading
              print line of column headings

       --ips  print ips instead of hostnames. with --lookup, canonicalizes
              based on stored IP, if available, rather than  stored  host-
              name

       -l, --login
              print system login processes

       --lookup
              attempt to canonicalize hostnames via DNS

       -m     only hostname and user associated with stdin

       -p, --process
              print active processes spawned by init

       -q, --count
              all login names and number of users logged on

       -r, --runlevel
              print current runlevel

       -s, --short
              print only name, line, and time (default)
```

```
       -t, --time
              print last system clock change

       -T, -w, --mesg
              add user's message status as +, - or ?

       -u, --users
              list users logged in

       --message
              same as -T

       --writable
              same as -T

       --help display this help and exit

       --version
              output version information and exit

       If FILE is not specified, use /var/run/utmp.  /var/log/wtmp as FILE
       is common.  If ARG1 ARG2 given, -m presumed: 'am i' or 'mom  likes'
       are usual.

AUTHOR
       Written by Joseph Arceneaux, David MacKenzie, and Michael Stone.

REPORTING BUGS
       GNU  coreutils  online  help:  <https://www.gnu.org/software/core-
       utils/>
       Report  any  translation  bugs  to  <https://translationpro-
       ject.org/team/>

COPYRIGHT
       Copyright  ©  2020  Free Software Foundation, Inc.  License GPLv3+:
       GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
       This is free software: you are free to change and redistribute  it.
       There is NO WARRANTY, to the extent permitted by law.

SEE ALSO
       Full documentation <https://www.gnu.org/software/coreutils/who>
       or available locally via: info '(coreutils) who invocation'

telnet@SOMU:~$ pwd
/home/telnet
telnet@SOMU:~$ ls
1065  1107  somu  somu_head
telnet@SOMU:~$ ls -1
1065
1107
somu
somu_head
telnet@SOMU:~$ ls =-t
ls: cannot access '=-t': No such file or directory
telnet@SOMU:~$ ls -t
somu  somu_head  1107  1065
```

```
telnet@SOMU:~$ head somu
1
2
3
4
5
6
7
8
9
10
telnet@SOMU:~$ tail somu
6
7
8
9
10
11
12
13
14
15
```

```
telnet@SOMU:~$ ls -a
.               .bashrc      .profile                   1107
..              .lesshst     .sudo_as_admin_successful  somu
.bash_history   .local       .viminfo                   somu_head
.bash_logout    .motd_shown  1065
telnet@SOMU:~$ ls -d
.
telnet@SOMU:~$ ls -p
1065/  1107/  somu  somu_head
telnet@SOMU:~$ ls -u
somu  somu_head  1107  1065
telnet@SOMU:~$ cd 1107
telnet@SOMU:~/1107$ cd
telnet@SOMU:~$ mkdir somu
mkdir: cannot create directory ‘somu’: File exists
telnet@SOMU:~$ mkdir sekhar
telnet@SOMU:~$ rmdir sekhar
telnet@SOMU:~$ cat somu
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

```
telnet@SOMU:~$ nano sekhar
telnet@SOMU:~$ paste somu sekhar
1       hello
2       good
3       morning
4       how
5       are
6       you
7
8
9
10
11
12
13
14
15
telnet@SOMU:~$ wc somu
15 15 36 somu
telnet@SOMU:~$ chmod mode somu
chmod: invalid mode: ‘mode’
Try 'chmod --help' for more information.
telnet@SOMU:~$ chmod somu
chmod: missing operand after ‘somu’
Try 'chmod --help' for more information.
telnet@SOMU:~$ mv somu sekhar
```

Result:

Thus the Basics of Unix operating system , commands have been studied successfully.

Ex.No:2 **DATE:**

SHELL PROGRAMMING

2.a SUM OF TWO NUMBERS

Aim:

To find the sum of two given numbers.

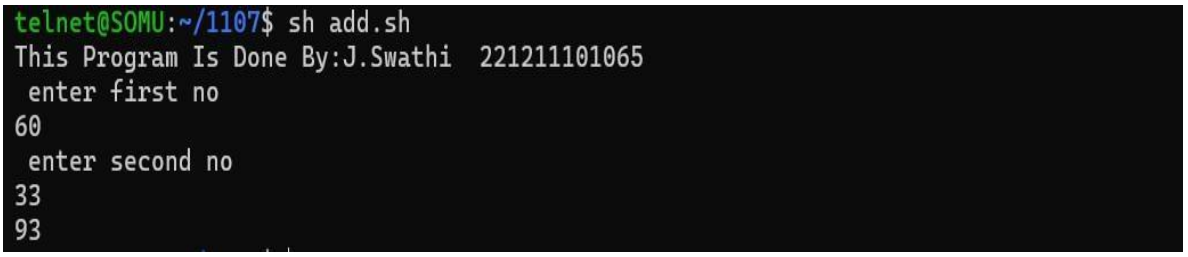
Algorithm:

1. Start
2. Declare variables a, b and sum.
3. Read values a and b.
4. Add the two numbers and assign the result to sum.
5. Display sum
6. Stop

Program:-

```
echo"This Program is Done By: J.Swathi \t 221211101065"
echo " enter first no "
read a
echo " enter second no "
read b
c= expr
$a + $b
```

Output:



2.b) FIBONACCI SERIES

Aim:

To find the Fibonacci series of the given number.

Algorithm:

1. Start
2. Declare variables first_term,second_term and temp.
3. Initialize variables first_term←0 second_term←1
4. Display first_term and second_term
5. Repeat the steps
    i. temp←second_term
    ii. second_term←second_term+first term
    iii. first_term←temp
    iv. Display second_term
6. Stop

Program:

```
echo"This Program is Done By: J.Swathi 221211101065"
echo "enter a number"
read n
i=0
a=0
b=1
c=0
echo "fibonacci series is"
echo "$a"
echo "$b"
n1=` expr $n - 2`
while [ $i -lt $n1 ] do
c=` expr $a + $b`
echo "$c"
a=$b
b=$c
i=` expr $i + 1`
done
```

Output:

```
telnet@SOMU:~/1065$ sh fibo.sh
This Program Is Done By: J.Swathi   221211101065
enter a number
9
fibonacci series is
0
1
1
2
3
5
8
13
21
```

2.c) POSITIVE OR NEGATIVE

Aim:

To find whether the given number is positive or negative.

Algorithm:

1. Start
2. Enter the value
3. Read the value
4. If Number is >0 Then Print “positive” else if Number is
5. Stop

Program:

```
echo"This Program is Done By: J.Swathi  221211101065"
echo " enter a number "
read a
if [ $a -eq 0 ]
then
echo " no is zero "
elif [ $a -gt 0 ]
then
echo " no is positive "
else
echo " no is negative "
fi
```

Output:

```
telnet@SOMU:~/1065$ sh positive.sh
 This Program is Done By:J.Swathi    221211101065
 enter a number
6
 no is positive
telnet@SOMU:~/1065$ sh positive.sh
 This Program is Done By:J.Swathi    221211101065
 enter a number
-8
 no is negative
telnet@SOMU:~/1065$ sh positive.sh
 This Program is Done By:J.Swathi    221211101065
 enter a number
0
 no is zero
```

2.d) GREATEST OF THREE NUMBERS

Aim:

To find the Greatest of the given three numbers.

Algorithm:

1. Start
2. Declare variables a,b and c.
3. Read variables a,b and c.
4. If a>b and If a>c
   Display a is the largest number.
   Else
   Display c is the largest number.
   Else
   If b>c
   Display b is the largest number. Else Display c is the greatest number.
5. Stop

Program:

```
echo"This Program is Done By: J.Swathi  221211101065"
echo " enter the three numbers "
read x
read y
read z
if [ $x -gt $y -a $x -gt $z ]
then
echo "$x is greater"
elif [ $y -gt $x -a $y -gt $z ]
then
echo "$y is greater"
else
echo "$z is greater"
fi
```

Output:

```
telnet@SOMU:~/1065$ sh greatest.sh
 This Program Is Done By: J.Swathi   221211101065
 enter the three numbers
34
88
99
99 is greater
```

2.e) ARMSTRONG NUMBER

Aim:

To find whether the given number is an Armstrong number or not.

Algorithm:

1. Start
2. Input the value N
3. Set SUM = 0
4. Number = N
5. Repeat
6. While Number ≠ 0
7. SUM = (Number%10)**3 + SUM Number = Number/10
8. If SUM == N then
9. Print N is an Armstrong Number.
10. Else
11. Print N is not an Armstrong Number.
12. Stop

Program:

```
echo"This Program is Done By: J.Swathi  221211101065"
echo "enter the number"
read num
ans=0
n=$num
while [ $n -gt 0 ]
do
q=` expr $n % 10`
ans=` expr $ans + $q \* $q \* $q`
n=` expr $n / 10`
done
if [ $ans -eq $num ]
then
echo "number is armstrong"
else
echo "number is not armstrong"
fi
```

Output:

```
telnet@SOMU:~/1065$ sh arm.sh
This Program Is Done By: J.Swathi     221211101065
enter the number
56
number is not armstrong
```

2.f) FACTORIAL NUMBER

Aim :

To write a program to generate the factorial of given number.

Algorithm:

1. Start
2. Declare variables n, fact and i.
3. Initialize variables fact←1 i←1
4. Read value of n
5. Repeat the steps until i=n 5.1: fact←fact* i 5.2: i←i+1
6. Display factorial
7. Stop.

Program:

```
echo"This Program is Done By: J.Swathi  221211101065"
n=0
fact=1
g=0
y=1
echo "enter no to find the factorial:"
read n
g=$n
while [ $n -ge $y ]
do
fact=` expr $fact \* $n`
n=` expr $n - 1 `
done
echo "factorial for $g is $fact"
```

Output:

```
telnet@SOMU:~/1065$ sh fact.sh
This Program Is Done By: J.Swathi   221211101065
enter no to find the factorial:
9
factorial for 9 is 362880
```

Result:

Thus the shell Programs were implemented and output was verified successfully.

Ex.No:3 **DATE:**

IMPLEMENTATION OF FILE SYSTEM

Aim:

To implement file allocation on free disk space in a contiguous manner.

Algorithm:

1. Start
2. Create a segment table that holds information about each segment.
3. Allocate memory for segments.
4. Translate a logical address to a physical address using the segment table.
5. Demonstrate segment allocation and address translation.

Program:

```
#include   <stdio.h>
#include  <conio.h>
#include <string.h>
int num=0, length[10], start[10];
char fid[20][4],a[20][4];
void directory()
{
int i;
printf("\nFile Start Length\n");
for(i=0;i<num;i++)
printf("%-4s %3d %6d\n",fid[i],start[i],length[i]);
void display()
{
int i;
for(i=0;i<20;i++)
printf("%4d",i);
printf("\n");
for(i=0;i<20;i++)
printf("%4s", a[i]);
```

```
}
void main()
{
int i,n,k,temp,st,l,ch,flag;
char id[4];
clrscr();
printf("This program is done by: J.Swathi \t 221211101065\n");
for(i=0;i<20;i++)
strcpy(a[i],"");
printf("Disk space before allocation:\n");
display();
do
{
printf("\nEnter Filename (max 3 char): ");
scanf("%s", id);
printf("Enter start block: ");
scanf("%d", &st);
printf("Enter no. of blocks: ");
scanf("%d", &l);
strcpy(fid[num],id);
length[num]=l;
flag = 0;
if((st+l) > 20)
{
printf("Requirement exceeds range\n");
continue;
}
for(i=st; i<(st+l); i++)
if(strcmp(a[i],"") !=0)
flag=1;
if(flag== 1)
{
printf("Contiguous allocation not possible.\n");
continue;
}
start[num] = st;
for(i=st;i<(st+);i++)
strcpy(a[i],id);
printf("Allocation donen");
num++;
```

```
printf("\nAny more allocation (1. yes / 2. no)?: ");
scanf("%d",&ch);
} while (ch==1);
printf("\n\t\t\tContiguous Allocation\n");
printf("Directory:");
Directory();
printf("\nDisk space after allocation:\n");
display();
printf("\n");
getch();
}
```

Output:-



Result:

Thus, the Implementation of file system was executed and output is verified successfully

Ex.No:4 **DATE:**

PROCESS CREATION

Aim:-

To write a C++ program to create a process using the fork system calls.

Algorithm:

1. Start the program.
2. Read the input from the command line.
3. Use fork() system call to create process, getppid() system call used to get the parent process ID and getpid() system call used to get the current process ID
4. If the pid > 0 Print parent and its process id otherwise print child and its Process id along with system date
5. Stop the program.

Program:-

```
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>
#include <unistd.h>
void main()
{
// make two process which run same
// program after this instruction
printf("This Work Done By:J.Swathi \t 221211101065\n");
pid_t p = fork();
if(p<0){
perror("fork fail");
exit(1);
}
printf("Hello world!, process_id(pid) = %d \n",getpid());
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi       221211101065
Hello world!, process_id(pid) = 1385
Hello world!, process_id(pid) = 1386
```

Result:

Thus the Program Process creation was implemented and output was verified successfully

Ex.No:5 **DATE:**

IMPLEMENTAION OF DINING PHILOSOPHER'S PROBLEM

Aim:

Write a C program to implement Dining Philosophers problem.

Algorithm:

1. Start
2. Declare the no.of free blocks(holes) and their sizes.
3. Get the size of the file to be loaded.
4. Have to allocate the first blocks that is big enough to hold the file.
5. Start search from the beginning of the set of holes.
6. If hole is large enough is found then stop searching and allocate the hole to the file
7. Otherwise display file that cannot be allocated

Program:

```
#include<stdio.h>
char state[10],self[10],spoon[10];
void test(int k)
{
if((state[(k+4)%5]!='e')&&(state[k]=='h')&&(state[(k+1)%5]!='e'))
{
state[k]='e';
self[k]='s';
spoon[k]='n';
spoon[(k+4)%5]='n';
}
}
void pickup(int i)
{
state[i]='h';
test(i);
if(state[i]=='h')
{
self[i]='w';
}
```

```
}
void putdown(int i)
{
state[i]='t';
spoon[i]='s';
spoon[i-1]='s';
test((i+4)%5);
test((i+1)%5);
}int main()
{
int ch,a,n,i;
printf("This Program Is Done By: J.Swathi \t 221211101065\n");
printf("\t\t Dining Philosopher Problem\n");
for(i=0;i<5;i++)
{
state[i]='t';
self[i]='s';
spoon[i]='s';
}
printf("\t\t Initial State of Each Philosopher\n");
printf("\n\t Philosopher No.\t Think/Eat \tStatus \tspoon");
for(i=0;i<5;i++)
{
printf("\n\t\t %d\t\t%c\t\t%c\t%c\n",i+1,state[i],self[i],spoon[i]);
}
printf("\n 1.Exit \n 2.Hungry\n3.Thinking\n");
printf("\n Enter your choice\n");
printf("\t\t");
scanf("%d",&ch);
while(ch!=1)
{
switch(ch)
{
case 2:
{
printf("\n\t Enter which philosopher is hungry\n");
printf("\t\t");
scanf("%d",&n);
n=n-1;
pickup(n);
```

```
break;
}
case 3:{
printf("\n\t Enter which philosopher is thinking\n");
printf("\t\t");
scanf("%d",&n);
n=n-1;
putdown(n);
break;
}
}
printf("\n\t State of Each philosopher\n\n");
printf("\n\t Philosoper No.\t Thinking\t Hungry");
for(i=0;i<5;i++)
{
printf("\n\t\t %d\t\t%c\t\t%c\t%c\n",i+1,state[i],self[i],spoon[i]);
}
printf("\n 1.Exit\n 2.Hungry\n 3.Thinking\n");
printf("\n Enter your choice\n");
printf("\t\t");
scanf("%d",&ch);
}
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065
                Dining Philosopher Problem
                Initial State of Each Philosopher

         Philosopher No.         Think/Eat      Status  spoon
                 1              t               s       s

                 2              t               s       s

                 3              t               s       s

                 4              t               s       s

                 5              t               s       s

 1.Exit
 2.Hungry
3.Thinking

 Enter your choice
                2

         Enter which philosopher is hungry
                5

         State of Each philosopher


         Philosoper No.  Thinking        Hungry
                 1              t               s       s

                 2              t               s       s

                 3              t               s       s

                 4              t               s       n

                 5              e               s       n

1.Exit
2.Hungry
3.Thinking

Enter your choice
                3

         Enter which philosopher is thinking
                3

         State of Each philosopher


         Philosoper No.  Thinking        Hungry
                 1              t               s       s

                 2              t               s       s

                 3              t               s       s

                 4              t               s       n

                 5              e               s       n

 1.Exit
 2.Hungry
 3.Thinking

 Enter your choice
                1
telnet@SOMU:~/1065$
```

Result:

Thus The Implementation of Dining Philosopher problem is implemented and output verified successfully.

Ex.No: 6 **DATE:**

INTERPROCESS COMMUNICATION USING SHARED MEMORY

Aim:

To implement Inter Process Communication using Message queue.

Algorithm:

1. Start the program
2. Use shmget() to allocate a shared memory
3. Use shmat() to attach a shared memory to an address space
4. Write the message in the shared memory area using the parent process
5. Read the message from the shared memory area using the child
6. Stop the program

Program:

SHARED MEMORY –SENDING

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/shm.h>
#include<string.h>
int main()
{
int i;
void *shared_memory;
char buff[100];
int shmid;
shmid=shmget((key_t)2345, 1024, 0666|IPC_CREAT);
printf("Key of shared memory is %d\n",shmid);
shared_memory=shmat(shmid,NULL,0);
printf("Process attached at %p\n",shared_memory);
printf("Enter some data to write to shared memory\n");
read(0,buff,100);
strcpy(shared_memory,buff);
printf("You wrote : %s \n",(char *)shared_memory);
}
```

SHARED MEMORY –RECEIVING

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/shm.h>
#include<string.h>
int main()
{
int i;
void *shared_memory;
char buff[100];
int shmid;
shmid=shmget((key_t)2345, 1024, 0666);
printf("Key of shared memory is %d\n",shmid);
shared_memory=shmat(shmid,NULL,0);
printf("Process attached at %p\n",shared_memory);
printf("Data read from shared memory is : %s\n",(char *)shared_memory);
}
```

Output:

```
Key of shared memory is 0
Process attached at 0x7f98eac4a000
Enter some data to write to shared memory
Hello Everyone This Is Swathi
You wrote : Hello Everyone This Is Swathi

telnet@SOMU:~/1065$ vi receive.c
telnet@SOMU:~/1065$ cc receive.c
telnet@SOMU:~/1065$ ./a.out
Key of shared memory is 0
Process attached at 0x7f3762f72000
Data read from shared memory is : Hello Everyone This Is Swathi
```

Result:

Thus the program Inter Process Communication using Shared memory segment is implemented and output verified successfully.

Ex.No: 7 **DATE:**

CPU SCHEDULING

7(a) FIRST COME FIRST SERVED CPU SCHEDULING

Aim:

To Implement CPU Scheduling Algorithms using first come first served

Algorithm:

1. Start the process.
2. Declare the array size.
3. Get the number of elements to be inserted.
4. Select the process that first arrived in the ready queue
5. Make the average waiting the length of next process.
6. Start with the first process from it's selection as above and let other process to be in queue.
7. Calculate the total number of burst time.
8. Display the values.
9. Stop the process.

Program:-

```
#include<stdio.h>
main()
{
int i,n,w[10],e[10],b[10];
float wa=0,ea=0;
printf("This Program is Done By: J.Swathi \t 221211101065\n");
printf("\nEnter the no of jobs: ");
scanf("%d",&n);
for(i=0;i<n;i++)
{
printf("\n Enter the burst time of job %d :",i+1);
scanf("%d",&b[i]);
if(i==0)
{
w[0]=0;
e[0]=b[0];
}
else
```

```
{
e[i]=e[i-1]+b[i];
w[i]=e[i-1];
}
}
printf("\n\n\tJobs\tWaiting time \tBursttime\tExecution time\n");
printf("\t-----------------------------------------------------------\n");
for(i=0;i<n;i++)
{
printf("\t%d\t\t%d\t\t%d\t\t%d\n",i+1,w[i],b[i],e[i]);
wa+=w[i];
ea+=e[i];
}
wa=wa/n;
ea=ea/n;
printf("\n\nAverage waiting time is :%2.2f ms\n",wa);
printf("\nAverage execution time is:%2.2f ms\n\n",ea);
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065

Enter the no of jobs: 4

 Enter the burst time of job 1 :8

 Enter the burst time of job 2 :4

 Enter the burst time of job 3 :9

 Enter the burst time of job 4 :7


        Jobs    Waiting time    Bursttime       Execution time
        ---------------------------------------------------------
        1               0               8               8
        2               8               4               12
        3               12              9               21
        4               21              7               28


Average waiting time is :10.25 ms

Average execution time is:17.25 ms
```

7(b) SHORTEST JOB FIRST CPU SCHEDULING

Aim:

To Implement CPU Scheduling Algorithms using shortest job first.

Algorithm:

1. Start the process.
2. Declare the array size.
3. Get the number of elements to be inserted.
4. Select the process which have shortest burst will execute first.
5. If two process have same burst length then FCFS scheduling algorithm used.
6. Make the average waiting the length of next process.
7. Start with the first process from it's selection as above and let other process to be in queue.
8. Calculate the total number of burst time.
9. Display the values.
10. Stop the process.

Program:

```
#include<stdio.h>
struct sjfs
{
char pname[10];
int btime;
}proc[10],a;
void main( )
{
struct sjfs proc[10];
int n,i,j;
int temp=0,temp1=0,temp2;
char name[20];
float tt,awt;
printf("This Program Is Done By: J.Swathi \t 221211101065\n");
printf("Enter the number of processes:\n");
scanf("%d", &n);
for(i=0;i<n;i++)
{
printf("\n Enter the process name: ");
```

```
scanf("%s",&proc[i].pname);
printf("\n Enter the Burst time:");
scanf("%d",&proc[i].btime);
}
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
{
if(proc[i].btime<proc[j].btime)
{
a=proc[i];
proc[i]=proc[j];
proc[j]=a;
}
}
}
printf("---------- CPU SCHEDULING ALGORITHM - SJFS ----
---------- ");
printf("\n\tprocess name \tBurst time \twaiting time \tturnaround time\n");
temp=0;
for(i=0;i<n;i++)
{
temp=temp1+temp+proc[i].btime;
temp1=temp1+proc[i].btime;
temp2=temp1-proc[i].btime;
printf("\n\t %s \t\t %d ms \t\t %d ms \t\t %d
ms\n",proc[i].pname,proc[i].btime,temp2,temp1);
}
printf("-------------------------------------------------------------------------------
");
awt=(temp-temp1)/n;
tt=temp/n;
printf("\nThe Average Waiting time is %4.2f milliseconds\n",awt);
printf("\nThe Average Turnaround time is %4.2f",tt);
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065
Enter the number of processes:
5

 Enter the process name: p1

 Enter the Burst time:67

 Enter the process name: p2

 Enter the Burst time:89

 Enter the process name: p3

 Enter the Burst time:55

 Enter the process name: p4

 Enter the Burst time:78

 Enter the process name: p5

 Enter the Burst time:67
-------------------- CPU SCHEDULING ALGORITHM - SJFS --------------------------
        process name    Burst time      waiting time    turnaround time

         p3      55 ms   0 ms    55 ms

         p1      67 ms   55 ms   122 ms

         p5      67 ms   122 ms          189 ms

         p4      78 ms   189 ms          267 ms

         p2      89 ms   267 ms          356 ms
-------------------------------------------------------------------------------
The Average Waiting time is 126.00 milliseconds

The Average Turnaround time is 197.00telnet@SOMU:~/1065$
```

7(c) ROUND ROBIN CPU SCHEDULING

Aim:

To Implement CPU Scheduling Algorithms using Round Robin

Algorithm:

1. Start the process.
2. Declare the array size.
3. Get the number of elements to be inserted.
4. Get the value.
5. Set the time sharing system with preemption.
6. Define quantum is defined from 10 to 100ms.
7. Declare the queue as a circular.
8. Make the CPU scheduler goes around the ready queue allocating CPU to each process for the time interval specified.
9. Make the CPU scheduler picks the first process and sets time to interrupt after quantum expired dispatches the process.
10. If the process has burst less than the time quantum than the process releases the CPU

Program:

```
#include<stdio.h>
#include<malloc.h>
void line(int i)
{
int j;
for(j=1; j<=i; j++)
printf("-");
printf("\n");
}
struct process
{
int p_id;
int etime, wtime, tatime;
};
struct process *p, *tmp;
int i, j, k, l, n, time_slice, ctime;
float awtime=0, atatime=0;
```

```
int main()
{
printf("This Program Is Done By: J.Swathi \t       221211101065\n");
printf("Process Scheduling - Round Robin \n");
line(29);
printf("Enter the no.of processes : ");
scanf("%d", &n);
printf("Enter the time slice : ");
scanf("%d", &time_slice);
printf("Enter the context switch time : ");
scanf("%d", &ctime);
p=(struct process*) calloc(n+1, sizeof(struct process));
tmp=(struct process*) calloc(n+1, sizeof(struct process));
for(i=1; i<=n; i++)
{
printf("Enter the execution time of process %d : ", i-1);
scanf("%d", &p[i].etime);
p[i].wtime=(time_slice+ctime)*i-1;
awtime += p[i].wtime;
p[i].p_id=i-1;
tmp[i]=p[i];
}
i=0; j=1; k=0;
while(i<n)
{
for(j=1; j<=n; j++)
{
if(tmp[j].etime <= time_slice && tmp[j].etime!=0)
{
k=k+tmp[j].etime;
tmp[j].etime=0;
p[j].tatime=k;
atatime += p[j].tatime;
k=k+ctime;
i++;
}
If(tmp[j].etime>time_slice && tmp[j].etime!=0)
{
k=k+time_slice+ctime;
tmp[j].etime -= time_slice;
```

```
}
}
}
awtime=awtime/n;
atatime=atatime/n;
printf("\nShedule \n");
line(60);
printf("Process\t\tExecution\tWait\t\tTurnaround\n");
printf("Id No\t\ttime\t\ttime\t\ttime\n");
line(60);
for(i=1;i<=n;i++)
printf("%7d\t%14d\t%8d\t%14d \n", p[i].p_id, p[i].etime, p[i].wtime,
p[i].tatime);
line(60);
printf("Avg waiting time :\t%2f \n",awtime);
printf("Avg turn around time :\t%2f\n",atatime);
line(60);
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065
Process Scheduling - Round Robin
-----------------------------
Enter the no.of processes : 3
Enter the time slice : 4
Enter the context switch time : 5
Enter the execution time of process 0 : 6
Enter the execution time of process 1 : 6
Enter the execution time of process 2 : 5

Shedule
------------------------------------------------------------
Process         Execution       Wait            Turnaround
Id No           time            time            time
------------------------------------------------------------
      0             6         8                   29
      1             6        17                   36
      2             5        26                   42
------------------------------------------------------------
Avg waiting time :      17.000000
Avg turn around time :  35.666668
------------------------------------------------------------
```

7(d) PRIORITY CPU SCHEDULING

Aim:

To Implement CPU Scheduling Algorithms using priority.

Algorithm:

1. Start the process.
2. Declare the array size.
3. Get the number of elements to be inserted.
4. Get the priority for each process and value
5. start with the higher priority process from it's initial position let other process to be queue.
6. Calculate the total number of burst time.
7. Display the values
8. Stop the process.

Program:

```
#include<stdio.h>
#include<malloc.h>
void line(int i)
{
int j;
for(j=1;j<=i;j++)
printf("-");
printf("\n");
}
struct process
{
int p_id,priority;
int etime,wtime,tatime;
};
struct process *p,temp;
int i,j,k,l,n;
float awtime=0,atatime=0;
int main()
{
printf("This Program Is Done By:J.Swathi \t 221211101065\n");
printf("Priority Scheduling\n");
line(29);
```

```
printf("Enter the no of processes : ");
scanf("%d",&n);
p=(struct process *) calloc(n+1,sizeof(struct process));
p[0].wtime=0;
p[0].tatime=0;
for(i=1;i<=n;i++)
{
printf("Enter the execution time of process %d:",i-1);
scanf("%d",&p[i].etime);
printf("Enter the priority of process %d:",i-1);
scanf("%d",&p[i].priority);
p[i].p_id=i;
}
for(i=1;i<=n;i++)
for(j=1;j<=n-i;j++)
if(p[i].priority>p[j+1].priority)
{
temp=p[j];
p[j]=p[j+1];
p[j+1]=temp;
}
for(i=1;i<=n;i++)
{
p[i].wtime=p[i-1].tatime;
p[i].tatime=p[i-1].tatime+p[i].etime;
awtime+=p[i].wtime;
atatime+=p[i].tatime;
}
awtime=awtime/n;
awtime=atatime/n;
printf("\nSchedule\n");
line(60);
printf("Process\t\texection\twait\t\tturnaround\n");
printf("Id No\t\t time\t\ttime\t\ttime\n");
line(60);
for(i=1;i<=n;i++)
printf("%7d\t%14d\t%8d\t%14d\n",p[i].p_id,p[i].etime,p[i].tatime);
line(60);
printf("Avg waiting time:\t %2f\n",awtime);
printf("Avg turnaround time:\t %2f\n",atatime);
```

```
line(60);
return(0);
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065
Priority Scheduling
----------------------------
Enter the no of processes : 4
Enter the execution time of process 0:8
Enter the priority of process 0:5
Enter the execution time of process 1:9
Enter the priority of process 1:2
Enter the execution time of process 2:6
Enter the priority of process 2:8
Enter the execution time of process 3:5
Enter the priority of process 3:3

Schedule
------------------------------------------------------------
Process         exection        wait            turnaround
Id No            time           time            time
------------------------------------------------------------
      1              8         8            1082309232
      2              9        17                    0
      3              6        23                    0
      4              5        28                    0
------------------------------------------------------------
Avg waiting time:        19.000000
Avg turnaround time:     76.000000
------------------------------------------------------------
```

Result:

Thus the Program Cpu Scheduling was implemented and output was verified successfully

Ex.No: 8 **DATE:**

IMPLEMENTATION OF BANKER'S ALGORITHM

Aim:

To write a C program to implement Bankers Algorithm.

Algorithm:

1. Start
2. Declare available(k) where is the instances of resource type , max[i,j]=k where process i request at most k instances of resource type j and allocation[i,j]=k where for process i k instances of resource type j are allocated
3. Compute Need[i,j]=Max[i,j]-allocation[i,j]
4. Let work and finish be vectors of length i and j respectively.
5. Initialize Work=available
6. if finish[i]=false for all processes
7. Find a process i such that *Finish* [*i*] = *false* and $Need_i$ □ *Work* and if no such i exists go to step 10
8. *Work* := *Work* + $Allocation_i$
9. If Finish[i]=true go to step 7
10. If finish[i]=true for all processes then display the system is in safe state else unsafe state

Program:

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
int i,j,z;
int res[10];
int resource,process,boolean=0;
int allocation[10][10],sel[10],max[10][10],need[10][10],work[10];
int check=0;
printf("This Program Is Done By: J.Swathi \t 221211101065\n");
```

```
printf("Welcome to Bankers Algorithms");
printf("\n Enter the no .of processes:");
scanf("%d",&process);
printf("\n Enter the number of Resources");
scanf("%d",&resource);
for(i=0;i<resource;i++)
{
printf("Enter the instances of Resource %d:",i+1);
scanf("%d",&res[i]);
}
for(i=0;i<process;i++)
{
printf("\n Enter allocated resources for process %d:",i+1);
for(j=0;j<resource;j++)
{
printf("\n Resource %d:",j+1);
scanf("%d",&allocation[i][j]);
}
}
for(i=0;i<process;i++)sel[i]=-1;
for(i=0;i<process;i++)
{
printf("Enter maximum need of process: %d",i+1);
for(j=0;j<resource;j++)
{
printf("\n Resource %d:",j+1);
scanf("%d",&max[i][j]);
}
}
for(i=0;i<process;i++)
for(j=0;j<resource;j++)
need[i][j]=max[i][j]-allocation[i][j];
printf("\n The Need is \n");
for(i=0;i<process;i++)
{
for(j=0;j<resource;j++)
{
printf("\t%d",need[i][j]);
}
printf("\n");
```

```
}
printf("\n The Avalilable is ");
for(i=0;i<process;i++)
{
work[i]=0;
for(j=0;j<resource;j++)
work[i]=work[i]+allocation[j][i];
work[i]=res[i]-work[i];
printf("\t%d",work[i]);
}
for(i=0;i<process;i++)
{
for(j=0;j<resource;j++)
{if(work[j]>=need[i][j]&&sel[i]==-1)
{
sel[i]=1;
work[j]=work[j]+allocation[i][j];
}
}}
for(i=0;i<process;i++)
{
if(sel[i]==1)
check=check+1;
else
{
check=-1;
}
}
if(check==process)
printf("\n System is in safe mode");
else
printf("\n System is in unsafe mode");
}
```

Output:

```
This Program Is Done By: J.Swathi        221211101065
Welcome to Bankers Algorithms
 Enter the no .of processes:4

 Enter the number of Resources3
Enter the instances of Resource 1:4
Enter the instances of Resource 2:6
Enter the instances of Resource 3:4

 Enter allocated resources for process 1:
 Resource 1:7

 Resource 2:8

 Resource 3:5

 Enter allocated resources for process 2:
 Resource 1:4

 Resource 2:6

 Resource 3:7

 Enter allocated resources for process 3:
 Resource 1:
5

 Resource 2:6

 Resource 3:7

 Enter allocated resources for process 4:
 Resource 1:8

 Resource 2:5

 Resource 3:3
Enter maximum need of process: 1
 Resource 1:8

 Resource 2:3

 Resource 3:2
Enter maximum need of process: 2
 Resource 1:1

 Resource 2:9

 Resource 3:5
Enter maximum need of process: 3
 Resource 1:4

 Resource 2:8

 Resource 3:4
Enter maximum need of process: 4
 Resource 1:8

 Resource 2:4

 Resource 3:3

 The Need is
        1       -5      -3
        -3      3       -2
        -1      2       -3
        0       -1      0

 The Avalilable is      -12     -14     -15     0
```

Result:

Thus the program Implementation of Banker's Algorithm was implemented and output verified successfully.

Ex.No: 9 **DATE:**

CONTIGUOUS MEMORY ALLOCATION

9(a) FIRST FIT MEMORY ALLOCATION

Aim:

To write a C++ program to implement First Fit Memory allocation technique.

Algorithm:

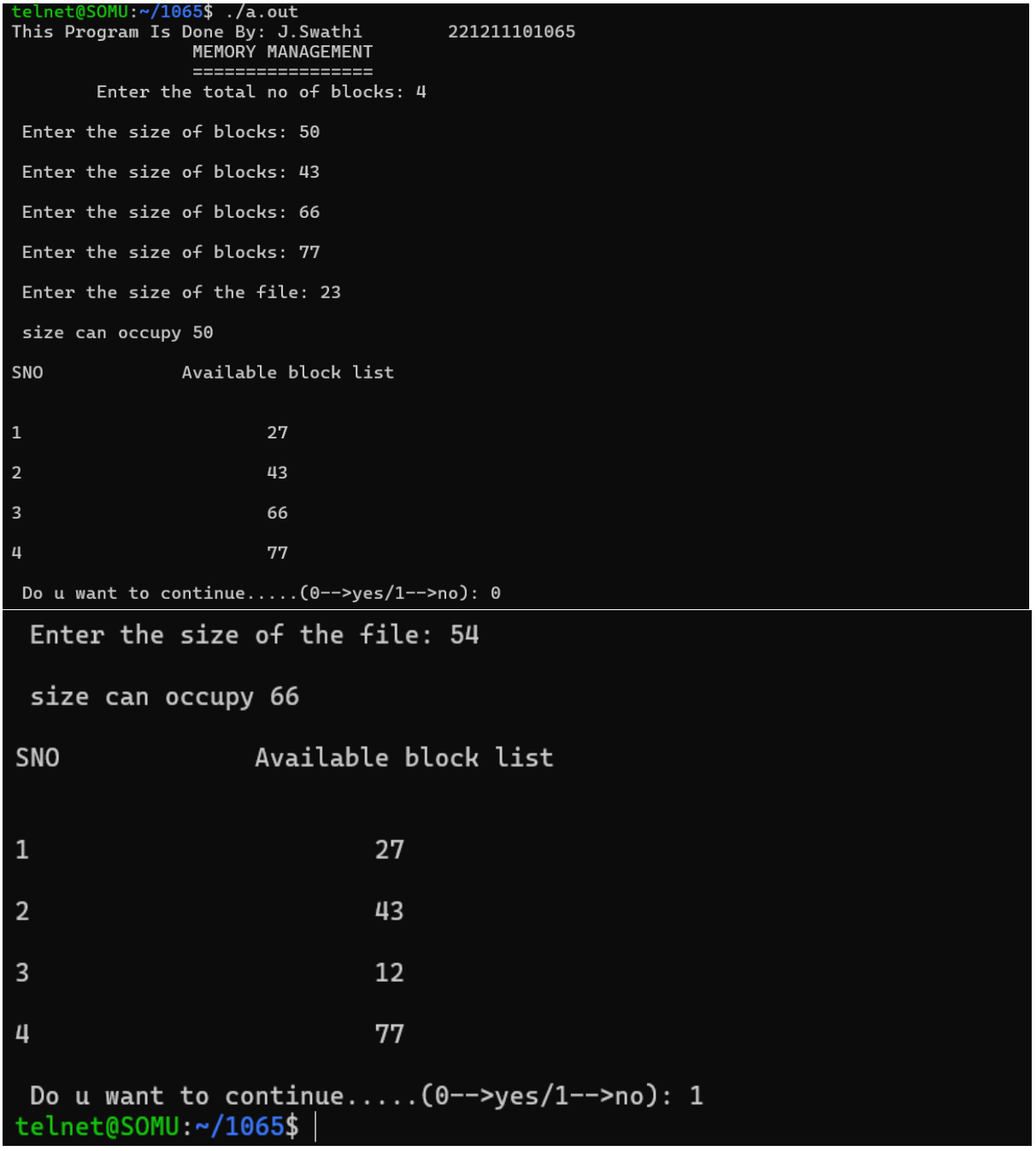
1. Start
2. Declare the no.of free blocks(holes) and their sizes
3. Get the size of the file to be loaded.
4. Have to allocate the first blocks that is big enough to hold the file.
5. Start search from the beginning of the set of holes.
6. If hole is large enough is found then stop searching and allocate the hole to the file
7. Otherwise display file that cannot be allocated
8. Stop

Program:

```
#include<stdio.h>
#include<string.h>
main()
{
int n,j,i,size[10],sub[10],f[10],m,x,ch,t;
int cho;
printf("\n This Program Is Done By: J.Swathi  \t 221211101065");
printf("\t\t MEMORY MANAGEMENT \n");
printf("\t\t  ================\n");
printf("\tEnter the total no of blocks: ");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
```

```
printf("\n Enter the size of blocks: ");
scanf("%d",&size[i]);
}
cho=0;
while(cho==0)
{
printf("\n Enter the size of the file: ");
scanf("%d",&m);
x=0;
for(i=1;i<=n;i++)
{
if(size[i]>=m)
{
printf("\n size can occupy %d",size[i]);
size[i]-=m;
x=i;
break;
}
}
if(x==0)
{
printf("\n\nBlock can't occupy\n\n");
}
printf("\n\nSNO\t\tAvailable block list\n") ;
for(i=1;i<=n;i++)
printf("\n\n%d\t\t\t%d",i,size[i]);
printf("\n\n Do u want to continue .... (0-->yes/1-->no): ");
scanf("%d",&cho);
}}
```

Output:


telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi 221211101065
MEMORY MANAGEMENT
Enter the total no of blocks: 4
Enter the size of blocks: 50
Enter the size of blocks: 43
Enter the size of blocks: 66
Enter the size of blocks: 77
Enter the size of the file: 23
size can occupy 50
SNO Available block list
1 27
2 43
3 66
4 77
Do u want to continue.....(0-->yes/1-->no): 0
Enter the size of the file: 54
size can occupy 66
SNO Available block list
1 27
2 43
3 12
4 77
Do u want to continue.....(0-->yes/1-->no): 1
telnet@SOMU:~/1065$

9(b) BEST FIT MEMORY ALLOCATION

Aim:

To write a C program to implement Best Fit Memory allocation technique

Algorithm:

1. Start
2. Declare the no.of free blocks(holes) and their sizes
3. Get the size of the file to be loaded.
4. Have to allocate the first blocks that is big enough to hold the file.
5. Start search from the beginning of the set of holes.
6. If hole is large enough is found then stop searching and allocate the hole to the file
7. Otherwise display file that cannot be allocated
8. Stop

Program:

```
#include<stdio.h>
main()
{
int n,j,i,size[10],sub[10],f[10],m,x,ch,t;
int cho;
printf("This Program Is Done By:J.Swathi \t 221211101065");
printf("\t\t MEMORY MANAGEMENT \n");
printf("\t\t ==================\n");
printf("\tENTER THE TOTAL NO OF blocks : ");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
printf("\n Enter the size of blocks : ");
scanf("%d",&size[i]);
}
cho=0;
while(cho==0)
{
printf("\n Enter the size of the file : ");
scanf("%d",&m);
for(i=1;i<=n;i++)
```

```
{
sub[i]=size[i];
if[i]=i;
}
for(i=1;i<=n;i++)
{
for(j=i+1;j<=n;j++)
if(sub[i]>sub[j])
{
t=sub[i];
sub[i]=sub[j];
sub[j]=t;
t=f[i];f[i]=f[j];
f[j]=t;
}
}
for(i=1;i<=n;i++)
{
if(size[f[i]]>=m)
{
printf("size can occupy %d : ",size[f[i]]);
size[f[i]]-=m;
x=i;
break;
}
}
 if(x==0)
{
printf("block can't occupy");
}
printf("\n\nSNO\t\t Available Block size\n") ;
for(i=1;i<=n;i++)
printf("\n%d\t\t%d",i,size[i]);
printf("\n\n Do u want to continue .... (0-->yes\t/1-->no)");
scanf("%d",&cho);
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065
                MEMORY MANAGEMENT
                =================
        ENTER THE TOTAL NO OF blocks : 4

 Enter the size of blocks : 50

 Enter the size of blocks : 100

 Enter the size of blocks : 200

 Enter the size of blocks : 150

 Enter the size of the file : 95
size can occupy 100 :

SNO              Available Block size

1               50
2               5
3               200
4               150

 Do u want to continue.....(0-->yes     /1-->no)0
```

```
 Enter the size of the file : 95
size can occupy 150 :

SNO              Available Block size

1               50
2               5
3               200
4               55

 Do u want to continue.....(0-->yes     /1-->no)1
telnet@SOMU:~/1065$
```

9(c) WORST FIT MEMORY ALLOCATION

Aim:

To write a C program to implement Worst Fit Memory allocation technique

Algorithm:

1. Start
2. Declare the no.of free blocks(holes) and their sizes
3. Get the size of the file to be loaded.
4. Have to allocate the first blocks that is big enough to hold the file.
5. Start search from the beginning of the set of holes.
6. If hole is large enough is found then stop searching and allocate the hole to the file
7. Otherwise display file that cannot be allocated
8. Stop

Program:

```
#include<stdio.h>
int p[10]={0},m=0,x=0,b[10]={0},a[10]={0};
main()
{
int j=0,n=0,pra[10]={0},pro[10]={0},z[10],ch,flag,flag2,sum=0,sum2=0;
int c=0,i=0,k=0;
printf("This Program Is Done By:J.Swathi \t 221211101065");
printf("\n\t\tMEMORY MANAGEMENT POLICIES\n");
printf("\n enter the no of process:\t");
scanf("%d",&n);
printf("\n enter the no of partition:\t");
scanf("%d",&m);
printf("\nprocess information\n");
for(i=0;i<n;i++)
{
printf("\n enter the memory required for process P%d:",i+1);
scanf("\t%d",&a[i]);
pro[i]=a[i];
}
printf("\n memory partition information\n");
for(j=0;j<m;j++)
```

```
{
printf("\n enter the block size of block B%d:",j+1);
scanf("\t%d",&p[j]);
pra[j]=p[j];
}
arrange();
printf("\n process partition\n\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
if(a[i]<p[j])
{
printf("%d\t%d\n",a[i],p[j]);
p[j]=0;
flag=i;
break;
}}
if(flag!=i)
printf("%d\t%s\n",a[i],"waiting");
arrange();
}
}
if(flag!=i)
printf("%d\t%s\n",a[i],"waiting");
arrange();
}
}
arrange()
{
int i,j,t;
for(i=0;i<m-1;i++)
for(j=0;j<m-i-1;j++)
{
if(p[j]<p[j+1])
{
t=p[j+1];
p[j+1]=p[j];
p[j]=t;
}}}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065

               MEMORY MANAGEMENT POLICIES

 enter the no of process:       5

 enter the no of partition:     5

process information

 enter the memory required for process P1:212

 enter the memory required for process P2:400

 enter the memory required for process P3:655

 enter the memory required for process P4:344

 enter the memory required for process P5:533

 memory partition information

 enter the block size of block B1:400

 enter the block size of block B2:688

 enter the block size of block B3:322

 enter the block size of block B4:788

 enter the block size of block B5:234

 process partition

212     788
400     688
655     waiting
344     400
533     waiting
telnet@SOMU:~/1065$
```

Result:

Thus the program Contiguous memory allocation technique was implemented and output verified

Ex.No: 10 **DATE:**

PAGE REPLACEMENT, PAGING AND SEGMENTATION

10(a) FIRST IN FIRST OUT PAGE REPLACEMENT

Aim:

To write a C Program to implement FIFO page replacement algorithm.

Algorithm:

1. Start
2. Declare the no.of blocks and no. of page references
3. If the referred page is in the main memory then refer it
4. If the referred page is not in main memory and free blocks are available then more the referred page to the existing free blocks.
5. Otherwise replace the first page that entered in main memory with the referred page and increment page fault
6. Stop

Program:

```
#include<stdio.h>
main()
{
int main_mem,cur=0,i=0,j,fault=0;
static int page[100],page_mem[100],flag,num;
for(i=0;i<100;i++)
page_mem[i]=-2;
printf("This Program Is Done By: J.Swathi \t 221211101065\n");
printf("\n\t\t\t\t paging->fifo\n");
printf("\n\n Enter the number of pages in main memory ");
scanf("%d",&main_mem);
printf("\n enter no of page references");
scanf("%d",&num);
for(i=0;i<num;i++)
{
printf("\n Enter page reference:");
```

```
scanf("%d",&page[i]);
}
printf("\n\t\t\t\t Fifo-> paging \n\n\n");
for(i=0;i<main_mem;i++)
printf("\t page %d",i+1);
for(i=0;i<num;i++)
{
for(j=0;j<main_mem;j++)
if(page[i]==page_mem[j])
{
flag=1;
break;
}
if(!flag)
{
page_mem[cur]=page[i];
fault++;
}
printf("\n\n");
for(j=0;j<main_mem;j++)
printf("\t%d",page_mem[j]);
if(!flag&&cur<main_mem-1)
cur++;
else if(!flag)
cur=0;
flag=0;
}
printf("\n\n-2 refers to empty blocks\n\n");
printf("\n\n No of page faults:%d\n",fault);
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065

                                paging->fifo


 Enter the number of pages in main memory 3

 enter no of page references6

 Enter page reference:8

 Enter page reference:5

 Enter page reference:9

 Enter page reference:4

 Enter page reference:7

 Enter page reference:3

                                Fifo-> paging


         page 1  page 2  page 3

        8       -2      -2

        8       5       -2

        8       5       9

        4       5       9

        4       7       9

        4       7       3

-2 refers to empty blocks


 No of page faults:6
```

10(b) LEAST RECENTLY USED PAGE REPLACEMENT

Aim:

To write a C Program to implement LRU page replacement algorithm.

Algorithm:

1. Start
2. Declare the no.of blocks and no. of page references
3. If the referred page is in the main memory then refer it
4. If the referred page is not in main memory and free blocks are available then more the referred page to the existing free blocks.
5. Otherwise replace the first page that entered in main memory with the referred page and increment page fault
6. Stop

Program:

```
#include<stdio.h>
#include<stdlib.h>
#define max 100
int frame[10],count[10],cstr[max],tot,nof,fault;
main()
{
getdata();
push();
}
getdata()
{
int pno,i=0;
printf("This Program Is Done By: J.Swathi \t 221211101065\n");
printf("\n\t\tL R U - Page Replacement Algorithm\n");
printf("\nEnter No. of Pages in main memory:");
scanf("%d",&nof);
printf("\nEnter the no of page references:\n");
scanf("%d",&pno);
for(i=0;i<pno;i++)
{
```

```
printf("Enter page reference%d:",i);
scanf("%d",&cstr[i]);
}
tot=i;
for(i=0;i<nof;i++)
printf("\tpage%d\t",i);
}
push()
{
int x,i,j,k,flag=0,fault=0,nc=0,mark=0,maximum,maxpos=-1;
for(i=0;i<nof;i++)
{
frame[i]=-1;
count[i]=mark--;
}
for(i=0;i<tot;i++)
{
flag=0;
x=cstr[i];
nc++;
for(j=0; j<nof; j++)
{
for(k=0; k<nof;k++)
count[k]++;
if(frame[j]==x)
{
flag=1;
count[j]=1;
break;
}
}
if(flag==0)
{
maximum = 0;
for(k=0;k<nof;k++)
{
if(count[k]>maximum && nc>nof)
{
maximum=count[k];
maxpos = k;
```

```
}
}
if(nc>nof)
{
frame[maxpos]=x;
count[maxpos]=1;
}
else
frame[nc-1]=x;
fault++;
dis();
}
}
printf("\nTotal Page Faults :%d",fault);
}
dis()
{
int i=0;
printf("\n\n");
while(i<nof)
{
printf("\t%d\t",frame[i]);
i++;
}
}
```

Output:

```
telnet@SOMU:~/1065$ ./a.out
This Program Is Done By: J.Swathi        221211101065

               L R U - Page Replacement Algorithm

Enter No. of Pages in main memory:3

Enter the no of page references:
8
Enter page reference0:8
Enter page reference1:3
Enter page reference2:6
Enter page reference3:3
Enter page reference4:9
Enter page reference5:2
Enter page reference6:8
Enter page reference7:4
        page0           page1           page2

        8               -1              -1

        8               3               -1

        8               3               6

        9               3               6

        9               3               2

        9               8               2

        4               8               2
Total Page Faults :7telnet@SOMU:~/1065$
```

10(c) IMPLEMENTATION OF PAGING

Aim:

To write a C Program to implementation of paging.

Algorithm:

1. Start
2. Create a page table that maps virtual pages to physical frames.
3. Mark all physical frames as free
4. Translate a virtual address to a physical address using the page table.
5. Load the required page into a free frame.
6. Demonstrate page allocation and address translation.
7. Stop

Program:

```
#include<stdio.h>
#include<conio.h>
struct ptable
{
int pno, staddr;
}p[30];
struct pcount
{
int pnum;
char data[30];
}pg[30][30];
void main()
{
int n,psize,i,j,k,st,pagenum,disp,phyaddr,laddr,found=0;
clrscr();
printf("This program is done by: J.Swathi \t 2221211101065\n");
printf("n Memory in formation ");
printf("\n Enter the no of pages: ");
scanf("%d",&n);
printf("Enter page size: ");
scanf("%d",&psize);
for(i=1;i<=n;i++)
```

```
{
printf("\n Enter the starting address for page[%d] ",i);
scanf("%d",&st);
for(j=1;j<psize;j++)
{
printf("Page[%d] data: ",st);
scanf("%s",&pg[i][j].data);
pg[i][j].pnum=st;
st=st+1;
}
}
printf(" \nMemory \n");
for(i=1;i<=n;i++)
{
for(j=1;j<=psize;j++)
{
printf("\n Page[%d] data: %s",pg[i][j].pnum,pg[i][j].data);
}
}
printf("\n\n Page table details \n");
printf("\n Enter the starting address for the page\n");
for(k=1;k<=n;k++)
{
printf("page[%d] starting address: ",k);
P[i].pno=k;
scanf("%d",&p[k].staddr);
}
printf("n Page number \t Starting address \n");
for(k=1;k<=n;k++)
printf("\n %d \t \t %d \n",p[k].pno,p[k].staddr);
printf("\n Enter the logical address: ");
scanf("%d",&laddr);
pagenum=laddr/100;
disp=laddr%100;
printf("\n pagenum: %d \t disp: %d ",pagenum,disp);
for(k=1;k<=n;k++)
{
if(p[k].pno==pagenum)
{
phyaddr=p[k].staddr+disp;
```

```
found=1;
break;
}
}
if(found==1)
{
printf("\n Physical address: %d "phyaddr);
for(j=1;j<=psize;j++)
{
if(pg[pagenum][j].pmum==phyaddr)
printf("\n Data: %s \n",pg[pagenum][j].data);
}
}
else
printf("\n Not Found\n");
getch();
}
```

Output:

```
telnet@SOMU:~/1107$ ./a.out
Page fault at page number: 0
Loaded page 0 into frame 0
Virtual address 0 -> Physical address 0
Page fault at page number: 2
Loaded page 2 into frame 1
Virtual address 8192 -> Physical address 4096
Page fault at page number: 4
Loaded page 4 into frame 2
Virtual address 16384 -> Physical address 8192
Page fault at page number: 6
Loaded page 6 into frame 3
Virtual address 24576 -> Physical address 12288
Page fault at page number: 8
Loaded page 8 into frame 4
Virtual address 32768 -> Physical address 16384
```

10(d) IMPLEMENTATION OF SEGMENTATION

Aim:

To write a C Program to implementation of segmentation concept.

Algorithm:

1. Start
2. Create a segment table that holds information about each segment.
3. Allocate memory for segments.
4. Translate a logical address to a physical address using the segment table.
5. Demonstrate segment allocation and address translation.

Program:

```
#include<stdio.h>
#include<conio.h>
struct ptable
{
int pno, staddr:
}p[30];
struct pcount
{
int pnum;
char data[30];
}pg[30][30]:
void main()
{
int n.psize,i,j,k,st,pagenum,disp,phyaddr,laddr,found=0;
clrscr();
printf("This program is done by: J.Swathi \t 221211101065\n");
printf("n Memory Information \n----------\n");
printf("n Enter the no of segments: ");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
printf("\n Enter the starting address and limit ofsegment[%d] ",i);
scanf("%d%d",&st, &psize);
```

```
for(j=1;j<=psize;j++)
{
printf("Page[%d] data: ",st);
scanf("%s",&pg[i][j].data);
Pg[i][j].pnum=st;
st=st+1;
}
}
printf("\nMemory Address\t DATA\n ----------\n");
for(i-1;i<=n;i++)
{
for(j=1;j<=psize;j++)
{
printf("\n %d\t\t%s"pg[i][j].pnum,pg[i][j].data);
}
}
printf("\n\n Segmentation Table Details \n----------\n");
printf("\n Enter the starting address for the page\n");
for(k=1;k<=n;kt+)
{
printf("page[%d] starting address: ",k);
p[i].pno=k;
scanf("%d",&p[k].staddr);
}
printf("n Page number \t Starting address\n");
for(k=1;k<=n;k++)
{
printf("\n %d\t\t %d\t\t%d \n",p[k].pno,p[k].staddr, psize);
}
printf("\n Enter the logical address: "):
scanf("%d",&laddr);
pagenum=laddr/100;
disp=laddr%100;
printf("n Segment no.: %d \n Disp: %d ",pagenum,disp);
for(k=1;k<=n;k++)
{
if(p[k].pno==pagenum)
{
phyaddr=p[k].staddr+disp;
found=1;
```

```
break;
}
}
if(found==1)
{
printf("\n Physical address: %d ",phyaddr);
for(j=1;j<=psize;j++)
{
if(pg[pagenum][i].pnum==phyaddr)
printf("\n Data: %s \n",pg[pagenum][j].data);
}
}
else
printf("\n Not found \n");
getch();
}
```

Output:-

```
telnet@SOMU:~/1107$ ./a.out
Logical address (Segment 0, Offset 100) -> Physical address 1100
Logical address (Segment 1, Offset 200) -> Physical address 2200
Logical address (Segment 2, Offset 150) -> Physical address 3150
Logical address (Segment 3, Offset 50) -> Physical address 4050
Logical address (Segment 4, Offset 90) -> Physical address 5090
Segment fault: offset 350 is out of bounds for segment 2
```

Result:

Thus the page replacement, Paging And Segmentation was implemented And output verified